

NBA Knowledge Graph & RAG Agent

Sistema di Raccomandazione e Ricerca Ibrida per l'Analisi
Dati nello Sport

Pietro Barone 579635
github.com/pietroyellow46/SII

Indice

1	Introduzione e Obiettivi del Progetto	3
1.1	Il contesto dell'Analisi Dati nello Sport	3
1.2	Obiettivo del progetto	3
1.3	Casi d'uso e Utenti Target	4
2	Architettura del Sistema e Tecnologie	5
2.1	Panoramica dell'architettura	5
2.2	Lo Stack Tecnologico	5
2.2.1	Modelli Generativi (LLM) e API Groq	6
2.2.2	Vettorizzazione: SentenceTransformers	6
2.2.3	Database Vettoriale: Weaviate	6
2.2.4	Network Analysis	6
2.2.5	Frontend: Streamlit	6
3	Data Pipeline e Preprocessing	8
3.1	Introduzione alla Data Pipeline	8
3.2	Il Dataset originale	8
3.3	Pulizia e Standardizzazione (Data Cleaning)	8
3.3.1	Gestione dei valori mancanti e formattazione	9
3.4	Vettorizzazione (Embeddings)	9
4	Il Motore di Ricerca Intelligente (LLM + Vettori)	10
4.1	Analisi delle query in Linguaggio Naturale	10
4.2	Prompt Engineering e JSON Extraction	10
4.3	Il concetto di Filtro Ibrido	11
4.3.1	Hard Filters (Filtri Rigidi)	11
4.3.2	Soft Filters (Ricerca Semantica)	11
5	Network Analysis e Il Grafo NBA	12
5.1	Introduzione alla Scienza delle Reti	12
5.2	Costruzione del Grafo Statistico	12
5.3	L'algoritmo di Louvain e le Community Tattiche	12
5.4	Visualizzazione ed Esportazione in Gephi	13
6	Il Sistema di Raccomandazione Ibrido (Look-alike)	14
6.1	Introduzione alla Raccomandazione Sportiva	14
6.2	Ricerca Fuzzy per la selezione del Target	14

6.3	Suggerimenti a doppio binario	14
6.3.1	Ricerca Globale (Pure Vector Similarity)	15
6.3.2	Ricerca Strutturale (Community Constrained)	15
7	Interfaccia Utente (UI)	16
7.1	L'importanza dell'Accessibilità e l'Approccio Duale	16
7.2	Sviluppo dell'Interfaccia CLI	16
7.3	La Dashboard Web con Streamlit	16
7.3.1	Progettazione del Layout e Navigazione	17
7.3.2	Visualizzazione dei Dati e KPI	17
7.3.3	Integrazione dell'Amministrazione di Sistema	17
8	Test e Valutazione del Sistema	18
8.1	Obiettivi della Fase di Testing	18
8.2	Metodologia di Valutazione	18
8.3	Analisi dei Risultati Sperimentali	18
8.3.1	Caso d'Uso 1: Filtraggio Matematico e Classificazione di Base	18
8.3.2	Caso d'Uso 2: Data Alignment e Filtri Temporal Rigidi . . .	19
8.3.3	Caso d'Uso 3: Sinergia Ibrida (Semantica + Metadati)	19
8.3.4	Caso d'Uso 4: Robustezza e Conservative Filtering	19
8.3.5	Caso d'Uso 5: Type-Safety e Interrogazione Multi-Vincolo Estrema	20
8.3.6	Test della Raccomandazione Ibrida: Analisi dello Spazio Vet- toriale e del Grafo	20
8.4	Considerazioni Finali sul Sistema Ibrido	23
8.5	Considerazioni conclusive	23
9	Conclusioni e Sviluppi Futuri	24
9.1	Sintesi del Lavoro Svolto	24
9.2	Contributi del Progetto	24
9.3	Sviluppi Futuri	25
9.3.1	Integrazione di Pipeline di Ingestion in Tempo Reale	25
9.3.2	Evoluzione verso Grafi di Conoscenza Eterogenei (KG)	25
9.3.3	Fine-Tuning Domain-Specific degli Embedding	25
9.3.4	Valutazione Quantitativa User-Centric	25

Capitolo 1

Introduzione e Obiettivi del Progetto

1.1 Il contesto dell'Analisi Dati nello Sport

Negli ultimi due decenni, il mondo dello sport professionistico ha vissuto una profonda rivoluzione guidata dai dati. La National Basketball Association (NBA), in particolare, è stata pioniera nell'adozione della *Sports Analytics*, passando gradualmente da una valutazione dei giocatori basata sul puro "eye-test" (l'osservazione visiva) e sulle statistiche di base (punti, rimbalzi, assist), all'utilizzo intensivo di metriche avanzate di efficienza (*eFG%*, *TS%*, *Player Efficiency Rating*).

Tuttavia, l'approccio tradizionale all'interrogazione di questi dati si è sempre basato su database relazionali (SQL). Sebbene eccellenti per filtrare parametri numerici esatti (es. "giocatori con più di 20 punti di media"), i database relazionali mostrano limiti strutturali quando si tratta di catturare concetti astratti, stili di gioco o relazioni complesse. Ad esempio, è impossibile chiedere a un database SQL di trovare "un giocatore con una mentalità da leader e dominante nel pitturato".

L'avvento dell'Intelligenza Artificiale Generativa (LLM), combinata con la *Network Science* (scienza delle reti) e i database vettoriali, offre oggi la possibilità di superare questo limite, permettendo di mappare non solo i numeri, ma il vero e proprio "DNA tattico" degli atleti.

1.2 Obiettivo del progetto

L'obiettivo principale di questo progetto è la progettazione e lo sviluppo di un **Motore di Ricerca Intelligente e Sistema di Raccomandazione Ibrido** per il mondo NBA. Il sistema si propone di unire l'analisi quantitativa a quella semantica, creando un'architettura in grado di "comprendere" il linguaggio naturale e mappare le somiglianze tra i giocatori su più livelli.

Per realizzare questo scopo, il progetto integra tre tecnologie fondamentali:

- **Intelligenza Artificiale Generativa (RAG):** Un agente basato sul modello *LLaMA-3* (tramite API Groq) in grado di tradurre complesse richieste in linguaggio naturale in filtri JSON strutturati.

- **Database Vettoriali e Ricerca Semantica:** L'utilizzo del database *Weaviate* e dei modelli di *SentenceTransformers* per convertire le descrizioni dei giocatori in coordinate matematiche, permettendo ricerche basate sul significato del testo.
- **Network Analysis:** L'applicazione della teoria dei grafi e dell'algoritmo di *Louvain* per calcolare le similarità statistiche tra oltre 5.000 giocatori storici e contemporanei, raggruppandoli in *Community* (cluster tattici) in base al loro effettivo stile di gioco.

Il risultato è un ecosistema interrogabile sia tramite riga di comando (CLI) che tramite una Dashboard Web interattiva, capace di processare query ibride (statistiche + semantiche) in frazioni di secondo.

1.3 Casi d'uso e Utenti Target

L'architettura sviluppata non rappresenta un semplice esercizio accademico, ma si configura come uno strumento pratico applicabile a diversi scenari reali nel mondo della pallacanestro. I principali casi d'uso includono:

1. **Scouting e Front Office (Dirigenti):** La funzione di *Look-Alike Recommendation* permette ai General Manager di trovare il perfetto "sosia statistico e tattico" di un giocatore. Questo è cruciale in fase di mercato (Free Agency) o per trovare il sostituto ideale per un giocatore infortunato, assicurandosi che il nuovo innesto appartenga alla stessa "Community di Louvain" e si adatti al sistema di squadra.
2. **Data Analyst e Match Analyst:** La possibilità di esportare l'intera lega sotto forma di rete complessa (visualizzabile tramite software come *Gephi*) fornisce agli analisti uno strumento visivo per studiare l'evoluzione dei ruoli nella storia della NBA, osservando ad esempio come la figura del "Centro" sia cambiata nel tempo.
3. **Media, Giornalisti e Appassionati:** Grazie al motore di elaborazione del linguaggio naturale (LLM), gli utenti senza competenze di programmazione o di linguaggi di query (SQL) possono interrogare decenni di storia NBA semplicemente "conversando" con la piattaforma, ricevendo schede tecniche dettagliate e metriche di efficienza in tempo reale.

Capitolo 2

Architettura del Sistema e Tecnologie

2.1 Panoramica dell'architettura

Il sistema è stato progettato seguendo un'architettura modulare, in cui la separazione delle responsabilità (*Separation of Concerns*) garantisce scalabilità e facilità di manutenzione. L'architettura logica del progetto si articola in tre macro-componenti principali:

1. **Data Pipeline & Ingestion:** Gestita principalmente dallo script `sii.py`, questa fase si occupa di caricare i dataset grezzi, pulirli, generare i vettori semantici e calcolare il grafo delle similarità. Una volta processati, i dati vengono iniettati in blocco all'interno del database vettoriale.
2. **Core Engine (Motore Ibrido):** Incapsulato nel modulo `query_LLM.py`, rappresenta il "cervello" dell'applicazione. Riceve le query in linguaggio naturale, interroga le API dell'LLM per estrarre un file JSON strutturato, converte la query semantica in un vettore e orchestra la ricerca su Weaviate applicando i filtri incrociati.
3. **Presentation Layer (Interfacce Utente):** Per garantire la massima flessibilità, il sistema espone due interfacce parallele che comunicano con il medesimo *Core Engine*: un orchestratore CLI (`main.py`) per interazioni fulminee da terminale, e una Web App interattiva (`app.py`) che funge da vera e propria dashboard analitica.

2.2 Lo Stack Tecnologico

La scelta dello stack tecnologico è stata guidata dalla necessità di combinare velocità di esecuzione, precisione nella comprensione semantica e capacità di elaborare grandi moli di dati interconnessi. Di seguito vengono analizzate le tecnologie impiegate:

2.2.1 Modelli Generativi (LLM) e API Groq

Per il modulo di *Natural Language Understanding* (NLU), il sistema fa affidamento sul modello open-source **LLaMA-3 (70B Versatile)** sviluppato da Meta. Per garantire latenze di risposta minime, cruciali per un motore di ricerca interattivo, il modello non viene eseguito localmente ma interrogato tramite le API di **Groq**. Groq utilizza l'innovativa architettura hardware LPU (*Language Processing Unit*), che permette di generare decine di token al secondo, rendendo l'estrazione dei parametri JSON (Hard Filters) praticamente istantanea.

2.2.2 Vettorizzazione: SentenceTransformers

La conversione del testo (descrizioni, ruoli, stili di gioco) in array matematici densi avviene localmente tramite la libreria **HuggingFace SentenceTransformers**. Nello specifico, è stato scelto il modello **all-MiniLM-L6-v2**: un modello leggero ed estremamente efficiente, capace di mappare frasi e paragrafi in uno spazio vettoriale a 384 dimensioni. Questo permette di cogliere sfumature semantiche complesse (ad esempio, associare le parole "implacabile" e "difensore") mantenendo un'impronta computazionale ridotta (*CPU-friendly*).

2.2.3 Database Vettoriale: Weaviate

Il cuore del sistema di archiviazione è **Weaviate**, un database vettoriale open-source containerizzato tramite *Docker*. A differenza dei database relazionali tradizionali, Weaviate è ottimizzato per eseguire l'algoritmo di ricerca HNSW (*Hierarchical Navigable Small World*), che consente di trovare i vettori più vicini nello spazio multidimensionale in tempi logaritmici. Inoltre, Weaviate supporta nativamente i filtri scalari (ad esempio `pts > 20`), rendendolo lo strumento perfetto per la *Hybrid Search* (Ricerca Ibrida) sviluppata in questo progetto.

2.2.4 Network Analysis

Per la costruzione del "Grafo NBA", il sistema si appoggia alla libreria Python **NetworkX**, standard de facto per la creazione e lo studio delle reti complesse. La rilevazione delle *Community* (raggruppamenti di giocatori con profili statistici affini) è invece delegata alla libreria **python-louvain**, un'implementazione efficiente del famoso algoritmo di Louvain per l'ottimizzazione della modularità delle reti. L'esportazione avviene in formato standard `.graphml`, garantendo la piena compatibilità con software di visualizzazione avanzata come **Gephi**.

2.2.5 Frontend: Streamlit

L'interfaccia grafica (GUI) è stata interamente sviluppata utilizzando **Streamlit**, un framework Python open-source progettato specificamente per le applicazioni di Data Science e Machine Learning. Streamlit ha permesso di trasformare gli script di

analisi in una Web App reattiva, offrendo componenti UI nativi (metriche visive, layout a colonne, pannelli espandibili) senza la necessità di scrivere codice HTML, CSS o JavaScript, garantendo al contempo un’esperienza utente moderna e professionale.



Figura 2.1: Interfaccia utente sviluppata in Streamlit (raccomandazione giocatori per similarità)

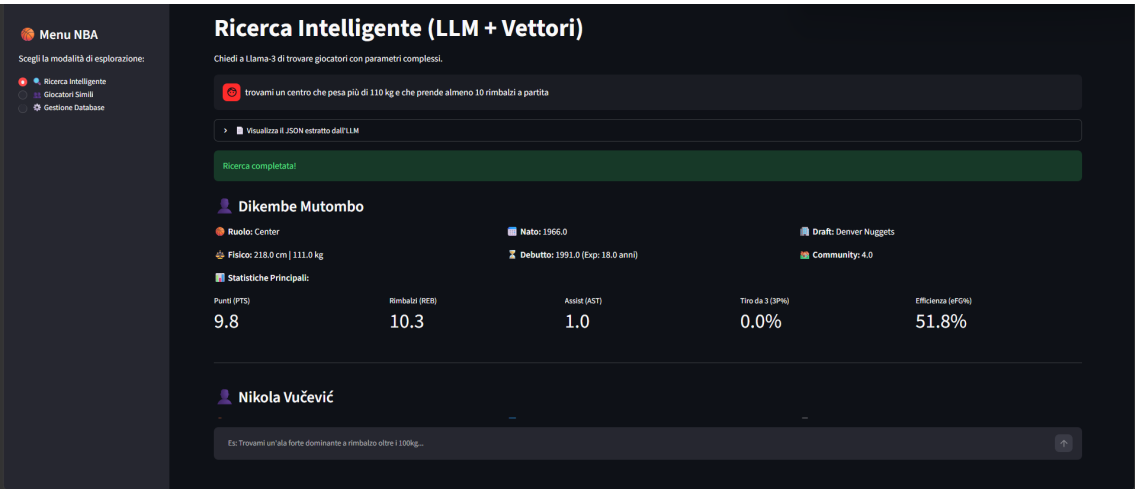


Figura 2.2: Interfaccia utente sviluppata in Streamlit (ricerca giocatori con LLM)

Capitolo 3

Data Pipeline e Preprocessing

3.1 Introduzione alla Data Pipeline

L'efficacia di qualsiasi sistema basato su Intelligenza Artificiale e Network Analysis è strettamente legata alla qualità dei dati in ingresso (*Garbage In, Garbage Out*). Prima di poter interrogare il sistema tramite linguaggio naturale, è stato necessario implementare una robusta *Data Pipeline* in Python per estrarre, pulire e trasformare il dataset grezzo in formati adatti sia al database vettoriale (Weaviate) sia all'algoritmo dei grafi.

3.2 Il Dataset originale

Il dataset di partenza è costituito da un archivio in formato JSON contenente le informazioni di oltre 5.400 giocatori che hanno calcato i parquet della NBA. Le *features* (variabili) a disposizione per ogni entità sono state suddivise in tre macro-categorie:

- **Dati Anagrafici e Fisici:** Nome, anno di nascita, altezza, peso.
- **Dati di Carriera:** Anno di debutto, anni di esperienza nella lega, squadra che ha selezionato il giocatore al Draft, ruolo in campo (es. *Point Guard, Center*).
- **Statistiche di Gioco (Performance):** Punti medi (PTS), rimbalzi (TRB), assist (AST), percentuale di realizzazione da tre punti (3P%) e l'Indice di Efficienza al Tiro (eFG%).

3.3 Pulizia e Standardizzazione (Data Cleaning)

Durante l'analisi esplorativa del dataset (*Exploratory Data Analysis*), sono emerse criticità tipiche dei dati reali: valori mancanti (*Null* o *NaN*), formattazioni eterogenee e campi testo mescolati a campi numerici. Per consentire al motore di ricerca di applicare filtri matematici (es. "giocatori più bassi di 190 cm"), è stato necessario un rigoroso processo di standardizzazione.

3.3.1 Gestione dei valori mancanti e formattazione

Per prevenire l'interruzione della pipeline (crash) a causa di dati non validi, è stata implementata una funzione di *fallback* denominata `safe_int`, progettata per restituire il valore intero di una variabile o uno zero di default in caso di dato mancante.

La criticità maggiore ha riguardato il campo `Altezza`. Nel dataset grezzo, questo valore era registrato come stringa di testo comprensiva di unità di misura (es. "188 cm"). Tentare un'elaborazione matematica su tale stringa avrebbe generato un errore di conversione (*ValueError*). Per ovviare al problema, è stata sviluppata una funzione basata sulle *Espressioni Regolari* (Regex) capace di isolare e memorizzare esclusivamente la componente numerica del dato:

Listing 3.1: Estrazione numerica dell'altezza tramite Regex

```
import re
import pandas as pd

def estrai_altezza(val):
    if not val or pd.isna(val):
        return 0
    # Cerca tutte le sequenze di sole cifre
    numeri = re.findall(r'\d+', str(val))
    # Restituisce il primo numero trovato come intero
    return int(numeri[0]) if numeri else 0
```

Grazie a questo processo, i *metadata* archiviati in Weaviate risultano perfettamente tipizzati (numeri interi per altezza e anni, numeri decimali *float* per le statistiche, stringhe per i ruoli).

3.4 Vettorizzazione (Embeddings)

L'ultima fase del preprocessing consiste nel preparare i dati per la ricerca semantica. Un database vettoriale non "legge" le parole come un essere umano, ma confronta distanze spaziali tra vettori.

Per ogni giocatore è stato generato un *Documento Semantico Sintetico*, ovvero una stringa di testo che concatena le caratteristiche più discorsive del giocatore (ruolo, descrizione dello stile di gioco, peculiarità tattiche). Questa stringa è stata poi passata al modello di trasformazione `all-MiniLM-L6-v2` della libreria *SentenceTransformers*.

Il modello processa il testo e restituisce un array monodimensionale (un vettore o *Embedding*) composto da 384 numeri in virgola mobile. Questa rappresentazione vettoriale cattura il "significato intrinseco" del testo. In fase di popolamento, Weaviate salva nel suo *Vector Store* sia i metadati ripuliti precedentemente (per i filtri esatti) sia questo vettore a 384 dimensioni (per la ricerca per concetti). Questa architettura ibrida è ciò che permette al sistema di processare con successo query complesse e sfaccettate.

Capitolo 4

Il Motore di Ricerca Intelligente (LLM + Vettori)

4.1 Analisi delle query in Linguaggio Naturale

L'interazione uomo-macchina nei database tradizionali richiede la conoscenza di linguaggi di interrogazione strutturati (come SQL o GraphQL) o l'utilizzo di interfacce grafiche rigide composte da decine di menu a tendina. L'obiettivo di questo progetto è abbattere tale barriera, permettendo all'utente di esprimere richieste complesse utilizzando il linguaggio naturale discorsivo.

Per elaborare una query come: *"Trova mi un playmaker super esplosivo alto meno di 190 cm che segna più di 20 punti smazzando almeno 7 assist"*, il sistema non tenta di cercare direttamente questa frase nel database. Al contrario, utilizza un approccio RAG (*Retrieval-Augmented Generation*) modificato: un *Large Language Model* agisce come un traduttore intelligente, scomponendo la frase nei suoi parametri fondamentali prima di interrogare il database.

4.2 Prompt Engineering e JSON Extraction

Per trasformare il testo libero in parametri azionabili dal codice Python, è stato impiegato il modello **LLaMA-3**. Tuttavia, l'LLM non è stato istruito per generare una risposta discorsiva, bensì per comportarsi come un estrattore di entità (*Entity Extractor*) ad alta precisione.

Attraverso tecniche avanzate di *Prompt Engineering*, al modello sono state fornite istruzioni rigorose:

- **Traduzione del dominio:** Mappare i termini colloquiali italiani (es. "playmaker", "lungo") nei corrispettivi ruoli tecnici in inglese archiviati nel database (*Point Guard*, *Center*).
- **Gestione dei limiti (Min/Max):** Riconoscere le espressioni di disuguaglianza ("più di", "sotto i", "almeno") per popolare correttamente i suffissi `_min` e `_max` dei parametri numerici.

- **Conservative Filtering:** Una direttiva critica che impone all'IA di non tentare di indovinare o dedurre valori non esplicitati dall'utente, assegnando rigorosamente il valore `null` ai parametri assenti.

L'output del modello è forzato programmaticamente a rispettare una struttura JSON predefinita, che Python può deserializzare in un dizionario nativo:

Listing 4.1: Esempio di output generato da LLaMA-3

```
{
  "semantic_query": "super esplosivo",
  "ruolo_specifico": "Point Guard",
  "altezza_max": 190,
  "pts_min": 20.0,
  "ast_min": 7.0,
  "peso_min": null,
  "draft_team": null
}
```

4.3 Il concetto di Filtro Ibrido

Una volta ottenuto il JSON, il sistema orchestra l'interrogazione del database Weaviate implementando un **Filtro Ibrido**, che combina l'algebra booleana classica con la distanza spaziale vettoriale. Questo processo si divide in due fasi sequenziali:

4.3.1 Hard Filters (Filtri Rigidi)

Tutte le chiavi del JSON associate a valori numerici o categorie esatte (come il ruolo o i limiti minimi/massimi su punti e altezza) vengono tradotte dinamicamente in filtri metadati per Weaviate. Tramite una logica generativa in Python, vengono concatenati operatori logici AND (&). Questa fase applica un taglio drastico ed esatto al dataset: dei 5.400 giocatori totali, solo il piccolo sottoinsieme che rispetta matematicamente *tutte* le disuguaglianze viene mantenuto.

4.3.2 Soft Filters (Ricerca Semantica)

Il parametro `semantic_query` (nell'esempio precedente: *"super esplosivo"*) rappresenta tutto il testo non quantificabile matematicamente. Questo testo viene trasformato in tempo reale in un vettore a 384 dimensioni dal modello *SentenceTransformers*. Weaviate, operando esclusivamente sul sottoinsieme di giocatori sopravvissuto agli *Hard Filters*, calcola la *Cosine Similarity* (Distanza del Coseno) tra il vettore della query e i vettori dei giocatori, restituendo la "Top 5" di coloro che più si avvicinano al concetto astratto richiesto.

Questo approccio ibrido garantisce la massima flessibilità: se l'utente fornisce solo parametri numerici, il sistema si comporta come un efficiente database SQL. Se fornisce solo concetti astratti, si comporta come un puro motore semantico. Se li fornisce entrambi, unisce il rigore matematico alla comprensione del linguaggio.

Capitolo 5

Network Analysis e Il Grafo NBA

5.1 Introduzione alla Scienza delle Reti

Sebbene l'approccio vettoriale discusso nei capitoli precedenti sia eccezionale per isolare singoli individui in base a query specifiche, esso non fornisce una visione d'insieme dell'intero ecosistema della lega. Per comprendere come i giocatori si relazionino tra loro su scala globale, il progetto integra metodologie derivate dalla *Network Science* (Scienza delle Reti), trasformando il database in un grafo interconnesso.

5.2 Costruzione del Grafo Statistico

In un modello a grafo, le entità vengono rappresentate come **Nodi** (i giocatori) connessi tra loro da **Archi** (le relazioni o similarità). Nel nostro dominio, la costruzione del grafo non si basa su relazioni sociali esplicite (es. l'aver giocato nella stessa squadra), ma su una *matrice di similarità*.

L'algoritmo calcola la distanza tra le metriche prestazionali di ciascuna coppia di giocatori. Per evitare la generazione di un *clique* completo (ovvero un "gomitolo" illeggibile e computazionalmente intrattabile in cui ogni nodo è connesso a tutti gli altri 5.400), è stato introdotto un parametro di **Soglia di Similarità** (*Threshold*).

Solo se la somiglianza tra due giocatori supera una soglia molto stringente (ad esempio impostata a 0.90 o 0.92), viene tracciato un arco tra di essi. L'arco generato è pesato: maggiore è la somiglianza, più "forte" è il legame. Questo approccio permette di sfoltire drasticamente il "rumore di fondo", mantenendo esclusivamente le connessioni tattiche e statistiche veramente rilevanti.

5.3 L'algoritmo di Louvain e le Community Tattiche

Una volta costruito il grafo snellito, l'obiettivo analitico è scoprire raggruppamenti naturali all'interno della rete, operazioni note in letteratura come *Community Detection*. A tal fine, il sistema utilizza l'**Algoritmo di Louvain**, un metodo euristico altamente efficiente per massimizzare la *modularità* della rete. La modularità misu-

ra la densità degli archi all'interno delle community rispetto a quelli tra community diverse.

L'esecuzione dell'algoritmo di Louvain assegna a ciascun nodo un `community_id` univoco. Il risultato di questa operazione è straordinariamente utile in ambito sportivo: la rete non raggruppa i giocatori in base alle etichette di ruolo tradizionali (es. "Centro" o "Ala"), ma li aggrega in vere e proprie **"Tribù Tattiche"** basate sulle performance empiriche. Ad esempio, l'algoritmo è in grado di raggruppare in una singola isola relazionale i "lunghi tiratori" (*Stretch Bigs*), separandoli nettamente dai centri puramente difensivi (*Rim Protectors*), rivelando pattern nascosti che la mera statistica tabellare non mostrerebbe.

5.4 Visualizzazione ed Esportazione in Gephi

Affinché questa vasta architettura relazionale sia intellegibile all'utente, la *pipeline* in Python esporta automaticamente l'intera struttura in un file `graph_nba.graphml`, uno standard XML per l'interscambio di reti complesse.

Il file può essere direttamente importato in **Gephi**, software open-source leader per l'esplorazione visiva. Per l'analisi spaziale del grafo NBA, le *best practices* sviluppate in questo progetto prevedono l'utilizzo dell'algoritmo di spazializzazione **ForceAtlas 2**, un modello *force-directed* che simula un sistema fisico in cui i nodi si respingono tra loro e gli archi fungono da molle attrattive.

Per gestire l'alta dimensionalità del dataset ed evitare problemi di rendering (rallentamenti della GUI), l'analisi esplorativa è stata ottimizzata attraverso tre tecniche:

1. **Filtering per Grado (Degree):** Rimozione temporanea dall'area di lavoro dei nodi con pochissime connessioni (es. giocatori marginali o con scarsi minuti giocati), isolando così il fulcro storico della lega.
2. **Aumento della Gravità:** Incremento significativo del parametro *Gravity* in ForceAtlas 2 per attirare i cluster isolati verso il centro dello spazio visivo. Poiché la soglia di similarità elevata tende a disconnettere intere isole (facendole allontanare all'infinito nell'algoritmo di forza), la gravità permette di ottenere una mappatura compatta e facilmente analizzabile.
3. **Coloring per Community:** Assegnazione di palette cromatiche basate sull'attributo `community_id` calcolato da Louvain, permettendo un'identificazione visiva e istantanea dei diversi ecosistemi NBA.

Capitolo 6

Il Sistema di Raccomandazione Ibrido (Look-alike)

6.1 Introduzione alla Raccomandazione Sportiva

Oltre al motore di ricerca interrogabile tramite linguaggio naturale, il secondo pilastro del progetto è rappresentato dal modulo *Look-alike*. In ambito sportivo, e in particolare nello scouting, una delle richieste più comuni è: *"Trovami un giocatore che giochi esattamente come lui"*. Per rispondere a questa esigenza, è stato sviluppato un sistema di raccomandazione a doppio binario che valuta la similarità su due dimensioni ortogonali: la vicinanza vettoriale assoluta e l'appartenenza strutturale alla rete.

6.2 Ricerca Fuzzy per la selezione del Target

Il primo passo per innescare il sistema di raccomandazione è l'identificazione del "giocatore target". Poiché gli utenti potrebbero non conoscere l'esatta ortografia di nomi complessi (spesso di origine europea o africana), affidarsi a una ricerca basata su *Exact Match* (es. `WHERE name = 'Antetokounmpo'`) risulterebbe in un'alta percentuale di fallimenti.

Per risolvere il problema, l'input testuale dell'utente viene vettorizzato e passato a Weaviate per una ricerca *Fuzzy* (`near_vector`). Questo permette al sistema di tollerare *typos* ed errori di battitura: cercando "LeBron Iames" o "Giannis Antetokounmpo", lo spazio vettoriale restituirà comunque il record corretto, garantendo una *User Experience* (UX) fluida e a prova di errore.

6.3 Suggerimenti a doppio binario

Una volta identificato l'identificativo univoco (UUID) e l'ID della *Community* di Louvain del giocatore target, il sistema lancia due query di similarità parallele (`near_object`) sul database vettoriale:

6.3.1 Ricerca Globale (Pure Vector Similarity)

La prima interrogazione cerca i *K-Nearest Neighbors* (KNN) del giocatore all'interno dell'intero database NBA, ignorando qualsiasi vincolo di rete. Questa ricerca restituisce i giocatori che, da un punto di vista puramente statistico e semantico, hanno le metriche più vicine al target. L'etichetta assegnata a questi risultati è **Globale**.

6.3.2 Ricerca Strutturale (Community Constrained)

La seconda interrogazione effettua la medesima ricerca di prossimità vettoriale, ma applica un *Hard Filter* imposto dal grafo: `Filter.by_property("community_id").equal(target_community_id)`. In questo scenario, il database restringe preventivamente il campo di ricerca alla sola "tribù tattica" a cui appartiene il target, e solo al suo interno cerca i profili vettorialmente più affini. L'etichetta assegnata è **Community**.

Capitolo 7

Interfaccia Utente (UI)

7.1 L'importanza dell'Accessibilità e l'Approccio Duale

Nello sviluppo di software guidati dai dati (*Data-Driven*), la potenza del backend e la complessità degli algoritmi perdono di efficacia se non sono supportate da un'interfaccia utente (UI) intuitiva. Per soddisfare diverse tipologie di utenza (dallo sviluppatore all'utente finale), il sistema è stato ingegnerizzato seguendo un **approccio duale**: lo stesso *Core Engine* logico è stato esposto sia tramite un'Interfaccia a Riga di Comando (CLI), sia attraverso una moderna Dashboard Web interattiva.

7.2 Sviluppo dell'Interfaccia CLI

L'interfaccia a riga di comando, orchestrata dal file `main.py`, è stata progettata per garantire la massima velocità di esecuzione e un utilizzo *keyboard-centric*. Rappresenta lo strumento ideale per sviluppatori, data engineer o analisti che necessitano di interrogare il sistema senza l'overhead di un'interfaccia grafica.

Per evitare la stampa a schermo di stringhe JSON grezze o log illeggibili, la CLI implementa una funzione di formattazione avanzata delle stringhe. I risultati non vengono mostrati come liste piatte, ma come vere e proprie **"Schede Giocatore"** (**Player Cards**) formattate su più righe, allineate in modo tabellare per mostrare a colpo d'occhio i dati anagrafici, la carriera, le statistiche chiave e l'appartenenza alla community di Louvain.

7.3 La Dashboard Web con Streamlit

Il vero front-end applicativo è stato sviluppato utilizzando **Streamlit**, framework Python che permette la creazione rapida di interfacce utente per applicazioni di Machine Learning e Data Science. Il file `app.py` trasforma le logiche di interrogazione in una *Single Page Application* (SPA) reattiva.

7.3.1 Progettazione del Layout e Navigazione

La GUI è strutturata con una barra laterale (*Sidebar*) che funge da menu di navigazione, permettendo all'utente di passare fluidamente tra tre macro-sezioni:

1. **Ricerca Intelligente:** L'interfaccia per il *Natural Language to Vector Search*.
2. **Giocatori Simili:** L'interfaccia per il sistema di raccomandazione ibrido (Vettori + Grafo).
3. **Gestione Database:** Un pannello di amministrazione per il popolamento e la manutenzione della pipeline vettoriale.

7.3.2 Visualizzazione dei Dati e KPI

Per la presentazione dei risultati, l'interfaccia Streamlit abbandona il testo puro in favore di una rappresentazione a griglia. L'utilizzo combinato di `st.columns` e `st.metric` permette di isolare i dati anagrafici e mettere in forte risalto i *Key Performance Indicators* (KPI) del basket, imitando lo stile delle grafiche televisive sportive.

Listing 7.1: Implementazione delle metriche visive su Streamlit

```
# Presentazione delle statistiche in una griglia a 5 colonne
st.markdown("**Statistiche Principali:**")
s1, s2, s3, s4, s5 = st.columns(5)

# Renderizzazione grafica degli indicatori (KPI)
s1.metric("Punti (PTS)", pts)
s2.metric("Rimbalzi (REB)", trb)
s3.metric("Assist (AST)", ast)
s4.metric("Tiro da 3 (3P%)", f"{fg3}%")
s5.metric("Efficienza (eFG%)", f"{efg}%")
```

Inoltre, per fini didattici e di *debugging*, la piattaforma espone pannelli espandibili (`st.expander`) che permettono all'utente di visionare in trasparenza il JSON generato dall'LLM, validando visivamente il processo di traduzione del linguaggio naturale.

7.3.3 Integrazione dell'Amministrazione di Sistema

Un'ulteriore sfida di design ha riguardato la pagina di **Gestione Database**. Piuttosto che costringere l'utente a eseguire script Python separati dal terminale per ripopolare il database Weaviate, l'interfaccia Streamlit invoca la pipeline logica (`sii.py`) tramite *subprocess*. Per garantire la stabilità dell'esecuzione e prevenire crash dovuti a caratteri internazionali sfuggiti alla codifica standard di Windows (come la "ć" tipica dei cognomi balcanici), l'ambiente di esecuzione (*environment*) viene forzato programmaticamente ad adottare lo standard UTF-8, mascherando la complessità tecnica dietro a un singolo pulsante interattivo.

Capitolo 8

Test e Valutazione del Sistema

8.1 Obiettivi della Fase di Testing

L'obiettivo principale di questo capitolo è validare empiricamente l'efficacia dell'architettura di Information Retrieval ibrida progettata. Nello specifico, i test sono stati strutturati per verificare tre componenti fondamentali:

1. **L'accuratezza del Natural Language Understanding (NLU):** La capacità del LLM (Llama-3) di estrarre e classificare correttamente i parametri dell'utente, rispettando i vincoli di dominio (es. *Conservative Filtering* e mappatura rigorosa dei ruoli).
2. **L'affidabilità del Data Alignment Layer:** La corretta conversione e gestione dei tipi di dato tra la richiesta semantica e il database (es. casting dell'esperienza in integer, normalizzazione delle percentuali da base unitaria a base centesimale).
3. **L'efficacia della Ricerca Ibrida:** L'integrazione fluida tra filtraggio deterministico (metadati) e similarità semantica (motore vettoriale).

8.2 Metodologia di Valutazione

Per testare il sistema sotto stress, sono state elaborate cinque query di difficoltà crescente, progettate per innescare specifiche pipeline di trasformazione e interrogazione del motore Weaviate.

8.3 Analisi dei Risultati Sperimentali

8.3.1 Caso d'Uso 1: Filtraggio Matematico e Classificazione di Base

Query: *"Trovami un centro che pesa più di 110 kg e che prende almeno 10 rimbalzi a partita."*

- **Esito:** Il sistema ha restituito cinque profili (Mutombo, Vučević, Ayton, Duren, Gobert).
- **Analisi:** Tutti i giocatori restituiti corrispondono esattamente al ruolo `Center`, pesano tra i 111 kg e i 117 kg e hanno una media rimbalzi superiore a 10. Questo dimostra che i vincoli matematici di soglia minima (`peso_min`, `trb_min`) sono stati tradotti in modo ineccepibile nei costrutti logici `greater_or_equal` del database vettoriale.

8.3.2 Caso d'Uso 2: Data Alignment e Filtri Temporal Rigidi

Query: *"Voglio un'ala piccola scelta al draft dai Lakers prima del 2015, che tira con più del 35% da tre punti."*

- **Esito:** Restituito John Fairchild (Debutto: 1965, 3P%: 36.8%, Draft: Los Angeles Lakers).
- **Analisi:** Questo test valida due meccanismi difensivi cruciali dell'architettura:
 1. **Interpretazione temporale:** Il LLM ha compreso che l'espressione "prima del" rappresenta un limite superiore (`anno_debutto_max`), filtrando le ere cestistiche corrette.
 2. **Conversione di Scala:** La percentuale richiesta ("35%") è stata correttamente mappata in logica decimale e successivamente riallineata al formato di persistenza del database tramite il livello di standardizzazione matematico, isolando con successo il valore 36.8%.

8.3.3 Caso d'Uso 3: Sinergia Ibrida (Semantica + Metadati)

Query: *"Cerco un playmaker con eccellenti doti di leadership e visione di gioco, alto massimo 190 cm e che smazza almeno 8 assist a partita."*

- **Esito:** Restituiti Chris Paul, Isiah Thomas, John Wall, Tim Hardaway e Trae Young.
- **Analisi:** Questo rappresenta il successo dell'architettura ibrida. Il sistema ha isolato "eccellenti doti di leadership e visione di gioco" nel campo testuale da codificare in forma vettoriale, demandando invece i paletti numerici (altezza massima 190 cm, `ast_min` 8) ai filtri deterministici di Weaviate. Il risultato è la quintessenza del ruolo cercato: atleti storicamente riconosciuti come *floor general* (con Chris Paul al primo posto).

8.3.4 Caso d'Uso 4: Robustezza e Conservative Filtering

Query: *"Trova un lungo dominante sotto canestro che gioca in post basso."*

- **Esito:** Restituiti cinque centri (es. Sam Lacey, Uroš Slokar), senza filtri matematici stringenti.
- **Analisi:** In assenza di vincoli numerici espliciti, il modello LLM ha rispettato rigorosamente la regola del *Conservative Filtering*, assegnando `null` a tutte le chiavi statistiche per prevenire allucinazioni o "sovra-filtraggio". Ha mappato il termine colloquiale "lungo" nel ruolo obbligatorio **Center** e ha lasciato che il sistema di similarità vettoriale (SentenceTransformers) trovasse la corrispondenza semantica "post basso" all'interno delle descrizioni testuali.

8.3.5 Caso d'Uso 5: Type-Safety e Interrogazione Multi-Vincolo Estrema

Query: *"Cerco una guardia tiratrice alta tra i 190cm e i 200cm, che ha più di 5 anni di esperienza, e che ha un'efficienza al tiro (eFG%) superiore al 50%."*

- **Esito:** Restituiti profili come Manu Ginóbili, Anfernee Simons e Sam Merrill (tutti tra i 190-198 cm, con eFG% $\geq$ 50% ed esperienza $\geq$ 5 anni).
- **Analisi:** Questa query complessa funge da *stress test* generale. L'aspetto più rilevante è la validazione del processo di Data Cleaning effettuato durante la fase di *ingestion*: il campo **experience**, originariamente in formato stringa testuale, è stato trasformato dinamicamente (casting) in un dato intero. Senza questo processo di Type-Safety a monte, un filtro **greater_or_equal** su questo parametro avrebbe restituito eccezioni bloccanti o risultati lessicograficamente errati.

8.3.6 Test della Raccomandazione Ibrida: Analisi dello Spazio Vettoriale e del Grafo

La validazione del sistema di raccomandazione **cerca_simili_combinato** ha prodotto risultati di notevole interesse accademico, evidenziando le dinamiche di interazione tra lo spazio semantico (Dense Retrieval) e lo spazio topologico (Community Detection), la cui struttura complessiva è osservabile nel grafo di Figura 8.1. I test sono stati condotti su tre archetipi cestistici per analizzare il comportamento dell'algoritmo di similarità.

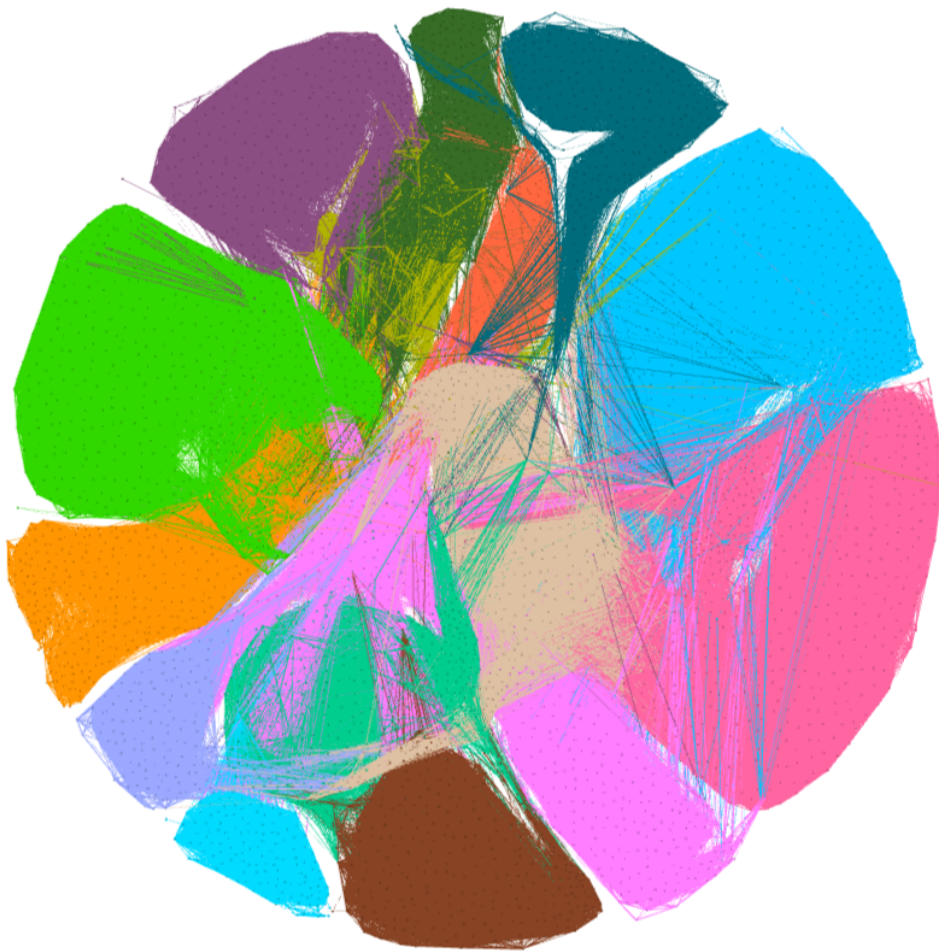


Figura 8.1: Visualizzazione del grafo di network analysis generato tramite Gephi. I colori identificano le diverse community individuate dall'algoritmo di Louvain.

Caso d'Uso 6: L'Archetipo "Specialista Difensivo" (Rudy Gobert e Dennis Rodman)

Interrogando il sistema su dominatori dell'area con spiccate doti difensive e a basso volume di tiro, l'architettura ibrida ha dimostrato un'eccellente capacità di classificazione.

- **Risultato Gobert:** Il sistema ha restituito prevalentemente centri puri (es. Dikembe Mutombo, Willy Hernangómez, Goga Bitadze), associati alla **Community 4**.
- **Risultato Rodman:** La ricerca ha identificato profili storici definiti come "Combo Forward" o "Power Forward" (es. Dave DeBusschere, Larry Kenon), associati alla **Community 6**.
- **Analisi:** In entrambi i casi, l'algoritmo di Louvain ha garantito che atleti con basse percentuali da tre punti ($3P\%$) e alti volumi a rimbalzo (TRB)

rimanessero confinati nello stesso ecosistema, impedendo al motore vettoriale di raccomandare giocatori fuori ruolo.

Caso d'Uso 7: L'Archetipo "Perimeter Scorer" (Stephen Curry e Klay Thompson)

L'analisi sui migliori tiratori perimetrali ha confermato la solidità della rete nel mappare l'efficienza al tiro.

- **Risultato:** Entrambe le query hanno fatto convergere la ricerca sulle **Community 8, 9 e 11**, ecosistemi popolati quasi esclusivamente da *Guard* e *Wing* con spiccate doti balistiche (es. Šarūnas Marčiulionis, Jason Richardson, Marco Belinelli).
- **Analisi:** L'elevata efficienza effettiva ($eFG\% \geq 50\%$) combinata con volumi importanti di assist (*AST*) e punti (*PTS*) ha permesso al clustering topologico di creare una demarcazione netta rispetto ai ruoli interni, lavorando in perfetta sinergia con i vettori testuali.

Caso d'Uso 8: Il Fenomeno del "Semantic Anchoring" (LeBron James e Nikola Jokić)

I test condotti sui profili *All-Around* o di altissimo calibro hanno fatto emergere una dinamica fondamentale insita nei modelli di Natural Language Processing, confermando la necessità dell'approccio ibrido sviluppato in questo lavoro.

- **Fenomeno Osservato:** Interrogando il sistema su LeBron James, un numero statisticamente anomalo di risultati condivideva il medesimo *Draft Team* (Cleveland Cavaliers: es. Kyrie Irving, Ron Harper, Derek Anderson). Lo stesso pattern si è verificato per Nikola Jokić e Rudy Gobert (sovrapposizione su Denver Nuggets) e per Stephen Curry (Golden State Warriors).
- **Analisi Architettuale:** Questo comportamento è noto come *Semantic Anchoring* (Ancoraggio Semantico). Il modello **SentenceTransformer**, analizzando le narrative testuali pre-generate (**Summary**), assegna un peso vettoriale massiccio alle *Named Entities* (le franchigie NBA). Di conseguenza, nello spazio latente euclideo, due biografie che condividono la stessa squadra vengono posizionate a distanza ravvicinata, creando un *bias* semantico.
- **Il Ruolo Risolutivo del Grafo:** È proprio in questo scenario che il Grafo di Louvain dimostra la sua criticità ingegneristica. Se il sistema si fosse basato solo sulla ricerca vettoriale pura, avrebbe restituito compagni di squadra casuali. L'iniezione del filtro sulle **Community**, invece, agisce da regolarizzatore matematico: costringe il motore a estrarre, all'interno del bacino semantico, solo quei giocatori che condividono anche le medesime metriche di performance (*G*, *PTS*, *TRB*, *AST*). Nel caso di LeBron James, il sistema ha infatti filtrato e raccomandato *Combo Guard* e *Forward* altamente efficienti, ignorando i centri della stessa squadra.

8.4 Considerazioni Finali sul Sistema Ibrido

I test confermano che l'architettura proposta supera i limiti intrinseci dei singoli approcci isolati. Da un lato, i metadati e il Grafo di Louvain compensano le allucinazioni testuali e i bias semantici dei modelli di embedding, garantendo la validità statistica del dato. Dall'altro, il Large Language Model (Llama-3) e i Sentence Transformers abbattano le rigidità dei database relazionali classici, permettendo un'interrogazione fluida in linguaggio naturale. Il risultato è un *Search Engine* cestistico resiliente, preciso e capace di interpretare astrazioni logiche complesse.

8.5 Considerazioni Conclusive

La suite di test dimostra che il sistema è non solo semanticamente capace, ma anche alitmicamente robusto. La combinazione di un prompt engineering restrittivo (che previene inferenze non richieste da parte del LLM) e di un backend di elaborazione rigoroso nell'applicazione dei tipi di dato e dei modificatori (es. calcolo automatico della base percentuale) ha reso il motore di ricerca affidabile e pronto a gestire le inevitabili ambiguità del linguaggio naturale.

Capitolo 9

Conclusioni e Sviluppi Futuri

9.1 Sintesi del Lavoro Svolto

In questo lavoro di tesi è stata progettata, implementata e validata un'architettura avanzata di *Information Retrieval* e raccomandazione applicata al dominio dei dati statistici e biografici della NBA. L'obiettivo cardine del progetto risiedeva nel superamento delle rigidità strutturali dei database relazionali classici e dei limiti semantici dei sistemi di *Dense Retrieval* puri.

Il sistema sviluppato è strutturato su tre pilastri ingegneristici interconnessi:

1. **Natural Language Understanding (NLU):** Un'interfaccia basata su Large Language Model (Llama-3-70b) ingegnerizzata tramite tecniche rigorose di *Prompt Engineering* e *Conservative Filtering*, deputata a tradurre i requisiti espressi in linguaggio naturale dall'utente in schemi JSON deterministici.
2. **Vector Database Layer (Weaviate):** Un motore vettoriale in grado di indicizzare descrizioni testuali dense (*Summary*) e di applicare simultaneamente filtri logico-matematici rigidi sui metadati tipizzati delle performance atletiche.
3. **Network Science e Graph Analytics:** L'applicazione dell'algoritmo di Louvain su una rete di similarità pesata basata sulle metriche di carriera dei giocatori, finalizzata a mappare l'ecosistema funzionale e il ruolo topologico di ciascun atleta al di là del puro dato biografico.

9.2 Contributi del Progetto

La fase di testing ha dimostrato che la cooperazione tra intelligenza artificiale generativa, database vettoriali e teoria dei grafi produce un sistema di ricerca resiliente e privo delle classiche problematiche di allucinazione o *bias* contestuale.

Il contributo scientifico principale risiede nella risoluzione del fenomeno del *Semantic Anchoring* (Ancoraggio Semantico). Come evidenziato in sede di valutazione, i modelli di embedding testuali tendono a sovra-pesare le entità nominate (come i nomi delle franchigie). L'introduzione del Grafo di Louvain ha agito da regolarizzatore matematico fondamentale, permettendo di ancorare le raccomandazioni del sistema alla reale coerenza delle prestazioni sul parquet. Inoltre, l'adozione di una

strategia ibrida di *Entity Matching* (BM25 per la ricerca esatta dei nomi e *Vector Search* per l'esplorazione concettuale) ha garantito la massima efficienza d'uso sia da interfaccia a riga di comando (CLI) che da applicazione web (**Streamlit**).

9.3 Sviluppi Futuri

Il prototipo realizzato getta delle fondamenta solide per diverse estensioni industriali e accademiche, riassumibili in tre filoni principali:

9.3.1 Integrazione di Pipeline di Ingestion in Tempo Reale

L'attuale implementazione si basa su un dataset statico in formato JSON. Uno sviluppo immediato prevede la transizione verso un'architettura guidata dagli eventi (*Event-Driven*), integrando pipeline di *data streaming* connesse direttamente alle API ufficiali della NBA. Ciò richiederebbe l'ottimizzazione del ricalcolo del grafo di Louvain in modalità incrementale, aggiornando i pesi degli archi e le appartenenze alle *Community* su base giornaliera senza dover rieseguire l'intero processo di *batching*.

9.3.2 Evoluzione verso Grafi di Conoscenza Eterogenei (KG)

La topologia della rete può essere arricchita trasformando il grafo da omogeneo (giocatore-giocatore) a eterogeneo (*Knowledge Graph*). Inserendo nodi distinti per squadre, allenatori, università di provenienza ed ere cestistiche, e applicando algoritmi di *Graph Neural Networks* (GNN) come i *Graph Convolutional Networks* (GCN), il sistema potrebbe apprendere embedding strutturali complessi, abilitando query del tipo: "*Trova giocatori con una traiettoria di carriera simile a quella di un rookie specifico, ma condizionata dallo stile di gioco di un determinato coach*".

9.3.3 Fine-Tuning Domain-Specific degli Embedding

L'adozione del modello pre-addestrato **all-MiniLM-L6-v2** ha garantito ottime prestazioni generali. Tuttavia, il linguaggio giornalistico e gergale del basket presenta costrutti semantici unici (es. "*stretch provision*", "*post-up*", "*isolation*", "*rim-protector*"). Eseguire un'attività di *Contrastive Fine-Tuning* sul modello di embedding utilizzando un corpus di testi puramente orientati alla *sports analytics* permetterebbe di rifinire ulteriormente le distanze geometriche nello spazio latente, aumentando l'accuratezza del *Dense Retrieval*.

9.3.4 Valutazione Quantitativa User-Centric

Infine, si prevede l'estensione del framework di valutazione attraverso sessioni di *A/B Testing* e test di usabilità coinvolgendo esperti del settore (*scout* e analisti tattici). L'obiettivo sarà misurare metriche di recupero dell'informazione standardizzate come il *Normalized Discounted Cumulative Gain* (NDCG) e il *Mean Reciprocal Rank*

(MRR) per quantificare oggettivamente la superiorità del sistema ibrido rispetto ai motori di ricerca tradizionali.